

Programación I – UTN TUPaD – Trabajo Practico Integrador

Gestión de Datos de Países en Python

Flores Vito, Lomoc Iván

Fecha de Entrega: 17/06/2026

Indice:

Marco Teorico	2
Decisiones Técnicas y Arquitectura	4
Dificultades y Conclusiones	8
Bibliografía y Webgrafía	9
Links de Maerial Solicitado	11

MARCO TEÓRICO

Introducción

El presente trabajo consiste en el desarrollo de una aplicación en Python para la gestión de países utilizando archivos CSV como mecanismo de persistencia de datos. El sistema permite agregar, actualizar, buscar, filtrar, ordenar y obtener estadísticas sobre distintos países almacenados en una base de datos simple.

Durante el desarrollo se aplicaron conceptos fundamentales de Programación I, tales como funciones, validación de datos, estructuras de datos basadas en diccionarios y listas, manejo de archivos CSV y control de excepciones.

Modularización

La modularización es una técnica de diseño de software que consiste en dividir un programa en componentes más pequeños e independientes llamados módulos. Cada módulo posee una responsabilidad específica y puede ser desarrollado, probado y mantenido de manera separada.

Entre las principales ventajas de la modularización se encuentran:

- Mayor organización del código.
- Reutilización de funciones.
- Facilidad de mantenimiento.
- Menor acoplamiento entre componentes.
- Mayor facilidad para detectar y corregir errores.

Ante el desconocimiento de cómo llevar a cabo correctamente esta práctica acudimos al siguiente material audiovisual externo al proveído por la institución, para así contrastar información y lograr un entendimiento a la hora de llevar el proyecto.

YouTube:

<https://www.youtube.com/watch?v=YCooYdrJZ2I>

https://www.youtube.com/watch?v=hWbD_6xhYe0&t=403s

Funciones

Las funciones permiten encapsular instrucciones bajo un nombre específico para reutilizarlas múltiples veces dentro de una aplicación.

Su utilización aporta:

- Reutilización de código.
- Reducción de duplicación.
- Mayor legibilidad.
- Facilita las pruebas y el mantenimiento.

Ejemplo utilizado en el proyecto:

- `agregar_pais()`
- `actualizar_pais()`
- `buscar_pais()`
- `filtrar_continente()`
- `estadisticas()`

Cada función recibe parámetros, realiza una tarea específica y devuelve resultados cuando es necesario.

Validación de Datos

La validación consiste en verificar que los datos ingresados por el usuario cumplan determinadas condiciones antes de ser procesados.

En el sistema se implementaron validaciones para:

- Nombres de países.
- Continentes válidos.
- Valores numéricos positivos.
- Rangos de población.
- Rangos de superficie.

Cuando una validación falla, las funciones retornan `None` para indicar que no se obtuvo un resultado válido. Esta técnica permite detectar errores y evitar operaciones incorrectas dentro del sistema.

Archivos CSV

CSV (Comma Separated Values) es un formato de almacenamiento de datos basado en texto plano donde la información se organiza en filas y columnas.

Se eligió este formato por:

- Su simplicidad.
- Fácil lectura por humanos.
- Compatibilidad con múltiples herramientas.
- Facilidad de manipulación mediante el módulo csv de Python.

Los datos almacenados incluyen:

- Nombre del país.
- Población.
- Superficie.
- Continente.

Manejo de Excepciones

El manejo de excepciones permite controlar errores durante la ejecución del programa sin provocar su finalización abrupta.

Se utilizaron bloques try/except para:

- Conversión de cadenas a números.
- Lectura de archivos.
- Validación de entradas incorrectas.

Esto mejora la robustez y estabilidad de la aplicación.

Persistencia de Datos

La persistencia de datos permite conservar información incluso después de finalizar la ejecución del programa.

En este proyecto la persistencia se implementó mediante archivos CSV, que son cargados al iniciar la aplicación y actualizados cada vez que se producen modificaciones en la información almacenada.

DECISIONES TÉCNICAS Y ARQUITECTURA

Arquitectura General del Sistema

El proyecto fue desarrollado siguiendo un enfoque modular, separando las distintas responsabilidades de la aplicación en archivos independientes. Esta decisión permitió mejorar la organización del código, facilitar el mantenimiento y reducir la complejidad del programa principal.

La aplicación se estructura en los siguientes módulos:

`main.py`

Es el punto de entrada del sistema. Su función principal es controlar el flujo general de ejecución, mostrar los menús, interpretar las opciones seleccionadas por el usuario y coordinar las llamadas a los distintos módulos.

Este archivo no contiene la lógica específica de cada operación, sino que delega las tareas a los módulos correspondientes.

`archivos.py`

Este módulo es responsable de la persistencia de datos.

Sus funciones principales son:

- Cargar la información almacenada en el archivo CSV al iniciar el programa.
- Guardar los cambios realizados por el usuario.
- Convertir los datos leídos desde el archivo a estructuras utilizables dentro del programa.

La utilización de este módulo permite centralizar todas las operaciones relacionadas con archivos en un único lugar.

`países.py`

Contiene las funciones relacionadas con la gestión de países.

Entre sus responsabilidades se encuentran:

- Agregar nuevos países.
- Actualizar información existente.
- Validar la existencia previa de un país antes de registrarlo.

Esta separación evita mezclar la lógica de negocio con la interfaz de usuario.

`filtros.py`

Agrupar todas las operaciones de búsqueda, filtrado y ordenamiento.

Permite:

- Buscar países por nombre.
- Filtrar por continente.
- Filtrar por población.
- Filtrar por superficie.
- Ordenar resultados según distintos criterios.

Al centralizar estas operaciones en un módulo específico, se facilita la incorporación de nuevos filtros en futuras versiones.

[estadisticas.py](#)

Implementa los cálculos estadísticos del sistema.

Entre ellos:

- País con mayor población.
- País con menor población.
- Promedio de población.
- Promedio de superficie.
- Cantidad de países por continente.

Esta decisión permite mantener aisladas las operaciones matemáticas del resto de la aplicación.

[validaciones.py](#)

Agrupar las funciones encargadas de verificar la validez de los datos ingresados por el usuario.

Entre las validaciones implementadas se encuentran:

- Validación de números enteros positivos.
- Validación de continentes permitidos.
- Validación de texto.

La existencia de un módulo exclusivo de validaciones evita duplicar código y permite reutilizar las mismas reglas en distintas partes del sistema.

[Estructura de Datos](#)

Se decidió almacenar la información utilizando una lista de diccionarios.

Cada país se representa mediante un diccionario con la siguiente estructura:

```
{  
  "nombre": "Argentina",  
  "poblacion": 46234830,  
  "superficie": 2780400,  
  "continente": "America"  
}
```

Esta estructura fue seleccionada debido a su simplicidad, facilidad de lectura y compatibilidad con el formato CSV utilizado para la persistencia de datos.

Persistencia de Datos

Para almacenar la información se eligió el formato CSV.

Las razones principales fueron:

- Facilidad de implementación.
- Compatibilidad con Python mediante el módulo csv.
- Facilidad para visualizar y modificar los datos manualmente.
- Adecuado para el volumen de información manejado por el proyecto.

La carga de datos se realiza al iniciar la aplicación y las modificaciones se guardan automáticamente después de cada operación que altera la información.

Problema Técnico Relevante Detectado

Durante las pruebas se detectó un inconveniente relacionado con la carga del archivo CSV.

Inicialmente el sistema parecía no reconocer varios países almacenados en la base de datos. Luego del proceso de depuración se comprobó que existían dos archivos llamados "países.csv" ubicados en directorios diferentes.

Python estaba utilizando una ruta relativa:

```
archivos.cargar_csv("países.csv")
```

lo que provocaba que el programa cargara un archivo distinto al esperado dependiendo del directorio desde el cual se ejecutaba.

La solución implementada consistió en construir una ruta absoluta utilizando:

```
os.path.dirname(__file__)  
os.path.join(...)
```

De esta forma se garantiza que siempre se utilice el archivo CSV perteneciente al proyecto independientemente del entorno de ejecución.

Diagrama General de Funcionamiento

Inicio del programa

|

Carga de datos desde CSV

|

Menú principal

|

Selección de operación

|-- Agregar país

|-- Actualizar país

|-- Buscar país

|-- Filtrar países

|-- Ordenar países

|-- Estadísticas

|

Guardar cambios (si corresponde)

|

Retorno al menú principal

|

Salida del sistema

DIFICULTADES Y CONCLUSIONES

Dificultades Encontradas

Durante el desarrollo surgieron distintos inconvenientes técnicos relacionados principalmente con la persistencia de datos y la organización del proyecto.

La dificultad más importante ocurrió durante las pruebas de búsqueda, filtrado y actualización de países. Inicialmente parecía que las funciones no reconocían los datos cargados en el archivo CSV.

Luego de realizar tareas de depuración mediante impresiones por consola (print), se descubrió que el programa estaba leyendo un archivo `paises.csv` diferente al esperado. Existían dos archivos con el mismo nombre en directorios distintos, por lo que Python cargaba información distinta a la que se visualizaba manualmente.

La solución consistió en utilizar rutas absolutas generadas mediante:

- `file`
- `os.path.dirname()`
- `os.path.join()`

Esto permitió garantizar que el programa siempre accediera al archivo CSV correcto independientemente del directorio desde el cual se ejecutara la aplicación.

Conclusiones

El proyecto permitió aplicar conocimientos fundamentales de Programación I mediante el desarrollo de una aplicación funcional basada en módulos.

Se comprendió la importancia de la modularización para organizar programas de tamaño mediano, facilitando su mantenimiento y escalabilidad.

Además, se adquirió experiencia práctica en:

- Manipulación de archivos CSV.
- Validación de datos.
- Uso de listas y diccionarios.
- Manejo de excepciones.
- Depuración de errores.
- Trabajo colaborativo utilizando Git y GitHub.

Como resultado, se logró construir una aplicación completa capaz de administrar información de países de forma persistente y estructurada.

Bibliografía/Webgrafía:

Documentacion Oficial UTN:

Funciones:

https://www.youtube.com/watch?v=SYfas1rP8js&list=PLOw7b-NX043aiYaKBMd3_k1IUZr8cXTzE&index=60

https://www.youtube.com/watch?v=hiRWPdVREFM&list=PLOw7b-NX043aiYaKBMd3_k1IUZr8cXTzE&index=61

https://tup.sied.utn.edu.ar/pluginfile.php/25771/mod_label/intro/5%20-%20Funciones.docx.pdf

<https://youtu.be/tsgcc5E9zEg?si=WRL0bQ93Sbei99qM>

<https://youtu.be/Sp0-fSNPrxI?si=XpdncpEmGb3Lq8tT>

<https://youtu.be/LC4SB2YAeu0>

<https://youtu.be/LfF3g0epM14>

https://tup.sied.utn.edu.ar/pluginfile.php/25774/mod_label/intro/Paso-por-Valor-y-Referencia-en-Python.pdf

<https://youtu.be/JqnClEskxLY?si=WAhP9VK-398FiwUu>

https://youtu.be/OBtl_hAMRhC?si=b5BBqHj3yy-mo4G2

https://tup.sied.utn.edu.ar/pluginfile.php/25778/mod_label/intro/alcances.pdf

Datos Complejos:

<https://www.youtube.com/watch?v=DNNjna9FW2Q>

<https://tup.sied.utn.edu.ar/mod/folder/view.php?id=11303>

<https://youtu.be/WQ1QYNOfvc>

<https://www.youtube.com/watch?v=5rznIMDlC68>

https://tup.sied.utn.edu.ar/pluginfile.php/25804/mod_label/intro/Diccionarios%20-%20TUPaD.docx%20%281%29.pdf

Manejo de errores:

https://www.youtube.com/watch?v=Sgs-fl_UisQ

https://tup.sied.utn.edu.ar/pluginfile.php/29375/mod_label/intro/Manejo%20de%20Errores_%20Tipo%20de%20Errores.docx.pdf

<https://www.youtube.com/watch?v=bQfoZKxUrGQ>

https://tup.sied.utn.edu.ar/pluginfile.php/29388/mod_label/intro/Manejo%20de%20errores_%20Try%20Except.docx.pdf?time=1771449850438

Manejo de archivos:

https://youtu.be/JpM9xU_gows?si=V3NEvxxvwn7HCyH_Z

https://drive.google.com/file/d/1PnAbJC65b5tkWhqn6oEZq4Q_aL_mdJLI/view?usp=drive_link

<https://www.youtube.com/watch?v=APpYlsVHSKs>

<https://www.youtube.com/watch?v=fH4WTL0WC9Y>

<https://youtu.be/yR7aHH-pHjg?si=qUBdJTdweuefL5LM>

<https://tup.sied.utn.edu.ar/mod/folder/view.php?id=15411>

<https://youtu.be/WvH8WpOByvc>

<https://youtu.be/sVkqK-JI-qs>

<https://tup.sied.utn.edu.ar/mod/folder/view.php?id=15420>

<https://youtu.be/7uYk0GwEJsU>

https://tup.sied.utn.edu.ar/pluginfile.php/25829/mod_label/intro/5%20-%20Manejo%20de%20errores%20con%20archivos.docx.pdf

Documentacion extraoficial:

Modularizacion:

<https://www.youtube.com/watch?v=YCooYdrJZ2I>

https://www.youtube.com/watch?v=hWbD_6xhYe0&t=403s

Links de material solicitado:

Video explicativo: <https://youtu.be/B6MIDQw-saE>

Repositorio GitHub: <https://github.com/vito-flores/TPI-Programacion1>